Submission Worksheet

IT202-450-M2026 / it202-milestone-2-2026

Submission Data

Course	IT202-450-M2026
Assignment	IT202 Milestone 2 - 2026
Student	Jason T. (jmt86)
Status	submitted
Worksheet Progress	100%
Potential Grade	10.00/10.00 (100.00%)
Received Grade	0.00/10.00 (0.00%)
Updated	7/27/2026, 6:45:18 PM

Grading Link
https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-2-2026/grading/jmt86

View Link
https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-2-2026/jmt86

Instructions

Verify that your lesson-built project data system is complete and functional, then collect focused evidence for the database, server-side API import, manual creation, admin list, public list, public details, edit, and delete requirements. Most coding should already be complete from the lessons; this worksheet focuses on confirming the full data flow, fixing remaining issues, and explaining your decisions.

Assignment Setup

- Required branch: `Milestone2`
- Required starting branch: `qa`
- Required project folder: `public_html/project/`
- Required deployed evidence target: working `qa` site before production promotion
- Recommended order:
  - Read the whole worksheet.
  - Review the modern project proposal and Milestone 2 assignment requirements.
  - Check out `qa`, pull the latest version, and create the milestone branch.
  - Ensure the related API, database-helper, and CRUD lessons are complete in your project.
  - Test each data source and CRUD behavior.
  - Merge the milestone branch into `qa`, test the deployed behavior, then collect worksheet evidence.

Git Workflow Checklist

- Check out `qa`.
- Pull the latest `qa` branch with `git pull origin qa`.
- Create the milestone branch with `git checkout -b Milestone2`.
- Commit focused verification and fixes in small pieces. If there is nothing to commit, wait until you add the PDF to your repository.
- Push `Milestone2`.
- Open a pull request from `Milestone2` into `qa` and keep it open while you work.

your repository.
Push `Milestone2`.
Open a pull request from `Milestone2` into `qa` and keep it open while you work.
After all Milestone 2 requirements pass locally, merge the pull request into `qa`.
Gather evidence from the deployed `qa` site and fill out the worksheet.
Export the worksheet PDF from the Learn Courses Platform.
Save the PDF in the repository under `docs/milestone02/`.
Add, commit, and push the PDF to the `Milestone2` branch.

```
git add .
git commit -m "Add milestone 2 worksheet PDF"
git push origin Milestone2
```

Upload the same PDF to Canvas.
Create and merge a pull request from `qa` into `prod` when production evidence is required.
Switch back to `qa`.
Pull the latest `qa` branch.

Shared Requirements

Apply these requirements to every Milestone 2 feature:

Add a comment with your UCID, date, and a brief summary of each feature area.
Load shared setup through `lib/app.php` and follow the course API, database, validation, flash, session, and role-check patterns.
Map API data in server-side PHP, keep secrets in the supported environment configuration, and store at least three useful API-derived values in real table columns.
Keep API-imported and manual records compatible in the same table shape when possible, clearly identify their source, and apply one duplicate-record rule consistently.
Include `id`, `created`, and `modified` on project data tables with reasonable types and constraints.
Validate form writes in HTML, the page's `validate()` function, and PHP; retain safe sticky values after an error.
Use prepared statements or shared database helpers, friendly flash messages, and `error_log()` for technical errors.
Enforce the authorized role on API import, manual creation, edit, and delete pages; hide management actions from users who cannot use them.
Keep navigation and styling consistent with the supported roles.
Test the supported deployed `qa` URLs before collecting evidence.
Use your own API, mapped data, database behavior, code, branch, pull request, URLs, and explanations.

Evidence Format

Most sections use the same evidence pattern:

Images:
- Show relevant code and matching browser or database output.
- Make sure UCID/date comments are visible in code evidence.
- Make sure the deployed `qa` URL is visible in browser evidence.
- One combined screenshot is acceptable when the code and output are both readable.
- Reuse earlier evidence when it already proves the same requirement.

URLs:
- Provide direct GitHub file links from the milestone branch.
- Provide anticipated production URLs for deployed PHP pages.
- Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
- PHP page URLs must end in `.php`; zero credit for that URL entry if it does not.

Response:
- Explain the logic in your own words.
- Describe the decisions made by your PHP, SQL, JavaScript, API mapping, or database design.

Response:

- Explain the logic in your own words.
- Describe the decisions made by your PHP, SQL, JavaScript, API mapping, or database design.
- Do not restate the prompt or list the requirements.

Submission Check

- Confirm the `Milestone2` branch and pull request into `qa` have been pushed.
- Confirm the deployed `qa` site was reviewed before collecting evidence.
- Test API import, manual creation, admin list, public list, public details, edit, and delete behavior.
- Test relevant filtering and sorting, the server-side limit rule, and friendly failure handling.
- Confirm anticipated production URLs use the tested `qa` paths with `-qa` changed to `-prod`.
- Commit the worksheet PDF under `docs/milestone02/` and upload the same PDF to Canvas.

Submission Progress 100%

Section #1: (2 pts.) Project Data Design

100%

Goal: show that the database stores useful mapped API data and compatible manual records.

Expected behavior:

- Project tables include `id`, `created`, and `modified`.
- At least three useful API-derived values use real columns with reasonable types and constraints.
- API and manual records share the same table shape whenever possible and can be distinguished.
- Nested or related API data is transformed into columns or related tables where appropriate.

Section #2: (2 pts.) Admin Api Import And Manual Create

100%

Goal: show that an authorized admin can import API records and create compatible manual records safely.

Expected behavior:

- Server-side PHP requests, validates, and maps API data before database writes.
- Manual creation uses HTML, JavaScript, and PHP validation.
- Duplicate API records follow one predictable rule.
- Server-side role checks protect both workflows.

Section #3: (1 pt.) Admin Data List

100%

Goal: show that an authorized admin can manage useful summaries of API and manual records.

Expected behavior:

- The page is protected by the authorized role.
- API-imported and manual records appear together as useful summaries.

Expected behavior:

- The page is protected by the authorized role.
- API-imported and manual records appear together as useful summaries.
- The list supports project-relevant filtering and sorting plus a server-validated limit from 1 through 100.
- Invalid limits default to 10, selected controls remain sticky, and empty results show friendly feedback.
- Each row links to the public detail page and includes the allowed edit and delete controls.

Section #4: (1 pt.) Public Data List

100%

Goal: show that non-admin users can browse useful record summaries without receiving management access.

Expected behavior:

- API-imported and manual records appear together with a link to the public detail page.
- The page supports project-relevant filtering and sorting plus a server-validated limit from 1 through 100.
- Invalid limits default to 10, selected controls remain sticky, and empty results show friendly feedback.
- Non-admin users do not see edit or delete controls.
- A shared role-aware list page is acceptable when it produces the same public and admin behavior.

Section #5: (1 pt.) Public Record View

100%

Goal: show a public detail page that safely displays one project record.

Expected behavior:

- The page validates the requested ID and shows more information than the list summary.
- A missing or invalid ID redirects with a friendly message.
- Non-admin users can view the detail without edit or delete controls.
- When the same page offers management actions to an authorized role, those actions appear only for that role.

Section #6: (1 pt.) Admin Edit Project Record

100%

Section #6: (1 pt.) Admin Edit Project Record

100%

Goal: show that an authorized admin can update approved record fields safely.

Expected behavior:

- The edit page validates the ID, preloads existing values, and validates submitted changes.
- System-managed and API identity/source fields remain protected unless the project intentionally allows them.
- Successful updates remain visible after refresh with friendly feedback.

Section #7: (1 pt.) Admin Delete Or Deactivate Project Record

100%

Goal: show that an authorized admin can safely remove or deactivate a record.

Expected behavior:

Delete uses POST, validates the record ID, and follows one consistent hard-delete or soft-delete rule.

The destination repeats the role check and redirects with friendly feedback.

Section #8: (1 pt.) Misc

67%

End of Assignment - submission has been recorded.